Game search

Minimax, and a real game, as a feed-forward network

This chapter turns the algorithm behind every board-game AI, **minimax**, into a network of matrices and ReLUs. Minimax answers one question: "if both sides play perfectly, what is this position worth to me, and which move should I make?" The recursion is two lines: a position where it is *my* turn is worth the **maximum** of its children; a position where it is *your* turn is worth the **minimum** of its children; and a finished game has a known score.

Three ideas carry the whole chapter.

1. **A tree level is one layer.** `max` and `min` are ReLU formulas (Chapter 1), so all nodes of a level are computed side by side by one matrix multiplication and one ReLU. A game that lasts D plies needs a fixed number of layers per ply (one in example 17, three in example 18), no matter how many nodes there are.
2. **Masks keep impossible moves out.** A real game has moves that are sometimes illegal, and positions where the game has already ended. The network cannot skip those branches, so it computes them anyway and uses 0/1 masks to make sure they can never win a `max` or a `min`.
3. **Sharing beats unrolling.** The tree for a pile of 12 has $3^{12} = 531441$ leaves, but only 13 *different* positions. A table of the 13 answers is a network with 15 neurons.

17 Minimax over a fixed tree

`basic_python/05_game_search/17_minimax_tree.py`

The problem

A game with a fixed shape. I move first and choose one of 3 moves; then you choose one of 2; then I choose one of 2; then the game ends and the leaf holds a score (higher is better for me). The 12 leaf scores are the input, each an integer in $[-3, 3]$. We want the value of the game and my best *first* move (moves are numbered 0, 1, 2; on ties take the lowest).

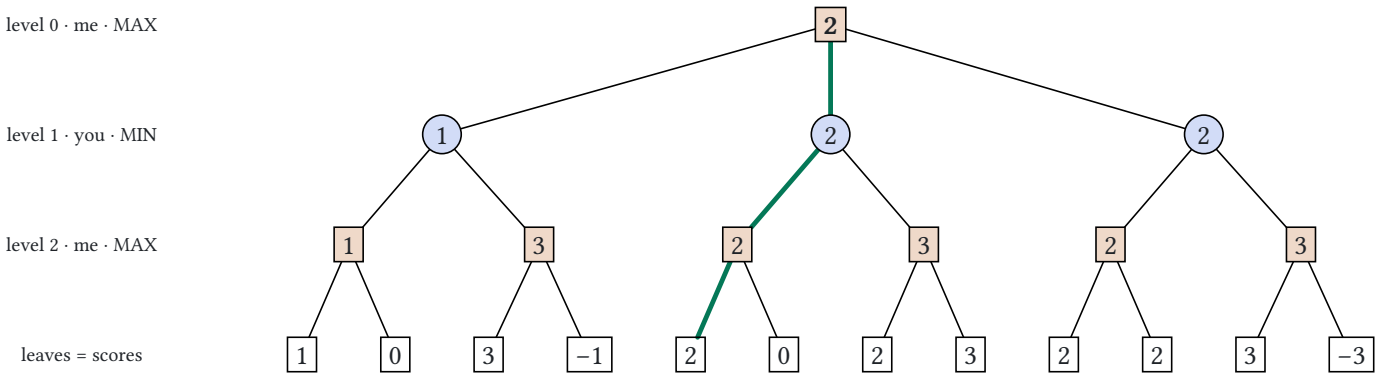


Figure 1: The tree for the example input. Each node shows its minimax value (squares: MAX of the children below, circles: MIN). The green path is the best play. Root moves 1 and 2 are tied at value 2; the lowest index wins.

Read it bottom-up. The left group of four leaves $(1, 0, 3, -1)$: I would pick $\max(1, 0) = 1$ or $\max(3, -1) = 3$ (level 2), and you, choosing between those two, would pick $\min(1, 3) = 1$ (level 1). So move 0 is worth 1 to me. Moves 1 and 2 are both worth 2, and the root takes the max: value 2.

Stage A versus Stage B

A · control flow

```
def minimax(leaves, depth=0, start=0):
    if depth == len(BRANCH):
        return leaves[start], None
    size = (leaves under one child)
    maxing = depth % 2 == 0
    best_v, best_m = None, None
    for m in range(BRANCH[depth]):
        v, _ = minimax(leaves, depth + 1,
                       start + m * size)
        if best_m is None or (
                v > best_v if maxing
                else v < best_v):
            best_v, best_m = v, m
    return best_v, best_m
```

Recursion, `if`, strict `>` so that the first of equal moves wins.

B · branchless arithmetic

```
def max2(a, b): return b + relu(a - b)
def min2(a, b): return a - relu(a - b)

def level_up(vals, branching, combine):
    out = []
    for g in range(len(vals) //
branching):
        acc = vals[g * branching]
        for i in range(1, branching):
            acc = combine(acc,
                vals[g * branching + i])
        out.append(acc)
    return out

v = level_up(leaves, 2, max2)  # 12 -> 6
v = level_up(v, 2, min2)       # 6 -> 3
# root: max2 over v, then argmax_first
```

No recursion, no `if`: one call per tree level, loops of fixed length.

Max and min are ReLU formulas

From Chapter 1: $\max(a, b) = b + \text{relu}(a - b)$ and $\min(a, b) = a - \text{relu}(a - b)$. If $a \le b$ then $\text{relu}(a - b) = 0$ and the formula returns b (for max) or a (for min); if $a > b$ then $\text{relu}(a - b) = a - b$ and it returns a or b. The kink of the ReLU sits exactly at $a = b$:

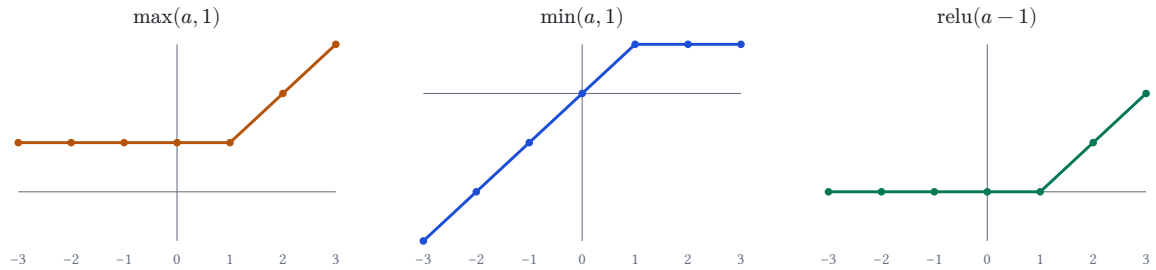


Figure 2: With $b = 1$: both are the identity line bent once at $a = 1$, and the bend is the ReLU $\text{relu}(a - 1)$.

One layer per level. All six pairs of leaves are independent, so we can do all six maxes at once. A single ReLU neuron holds only $\text{relu}(a - b)$; to also carry b forward through the ReLU we use $b = \text{relu}(b) - \text{relu}(-b)$, which is exact for negative b too (that is why leaves may be negative). So each pair gets three neurons, and the value is a plain sum of them:

$$\max(a, b) = \underbrace{\text{relu}(a - b)}_{n_1} + \underbrace{\text{relu}(b)}_{n_2} - \underbrace{\text{relu}(-b)}_{n_3}.$$

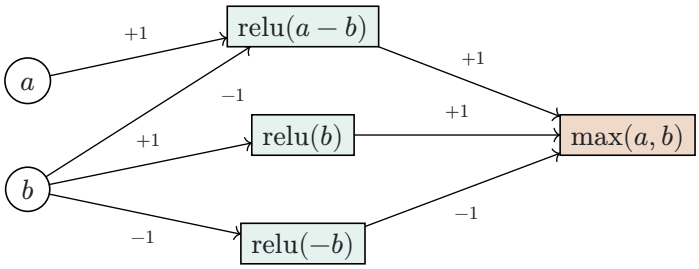


Figure 3: One "max gate". The hidden layer is the ReLU part; the last arrows are the (linear) readout, which is folded into the next layer's weights.

For `min`: $\min(a,b) = -\operatorname{relu}(a-b) + \operatorname{relu}(a) - \operatorname{relu}(-a)$. Here are the actual numbers for the example, level 2 (MAX of leaf pairs):

pair	a	b	$\operatorname{relu}(a-b)$	$\operatorname{relu}(b)$	$\operatorname{relu}(-b)$	max
0	1	0	1	0	0	1
1	3	−1	4	0	1	3
2	2	0	2	0	0	2
3	2	3	0	3	0	3
4	2	2	0	2	0	2
5	3	−3	6	0	3	3

Look at pair 1: $a = 3$, $b = -1$. $\operatorname{relu}(a-b) = \operatorname{relu}(4) = 4$, $\operatorname{relu}(b) = \operatorname{relu}(-1) = 0$ and $\operatorname{relu}(-b) = \operatorname{relu}(1) = 1$, so the sum is $4 + 0 - 1 = 3$. Correct. If we had only used $\operatorname{relu}(b)$, the $b = -1$ would have been lost (the ReLU clips it to 0), and the answer would have been 4.

Level 1 then takes the MIN of consecutive pairs of the six values $(1, 3, 2, 3, 2, 3)$, giving $(1, 2, 2)$.

The best move: argmax with the lowest index on ties

The three root children are worth $v = (1, 2, 2)$. We want a one-hot vector saying which is the best move, and when two are equal the *lower* index must win. This is the same idea as Chapter 3's argmax: compare, then AND.

Entry i wins exactly when it is **strictly** better than every earlier entry and **at least as good as** every later entry. Let $g_{ij} = [v_i \geq v_j]$ for $i < j$, which is $\operatorname{step}(v_i - v_j + 1)$. Then:

$$\begin{aligned} \text{win}_0 &= g_{01} \wedge g_{02} \\ \text{win}_1 &= (1 - g_{01}) \wedge g_{12} \\ \text{win}_2 &= (1 - g_{02}) \wedge (1 - g_{12}) \end{aligned}$$

and $a \wedge b = \operatorname{relu}(a + b - 1)$. Each comparison is two ReLUs ($\operatorname{step}(d) = \operatorname{relu}(d) - \operatorname{relu}(d-1)$). Numbers for $v = (1, 2, 2)$:

g_{01}	g_{02}	g_{12}	win_0	win_1	win_2
$\operatorname{step}(1-2+1) = 0$	$\operatorname{step}(1-2+1) = 0$	$\operatorname{step}(2-2+1) = 1$	$\operatorname{relu}(0+0-1) = 0$	$\operatorname{relu}(1+1-1) = 1$	$\operatorname{relu}(1+0-1) = 0$

Entries 1 and 2 tie, but $v_2 > v_1$ is false (it is a tie), so entry 2's third AND fails and entry 1 stays the only winner. The move number is then an *affine decode* of the one-hot: $\text{move} = 0 \cdot \text{win}_0 + 1 \cdot \text{win}_1 + 2 \cdot \text{win}_2 = 1$.

Stage C: the weights

Four ReLU layers. Every pair of siblings uses the same small block; `np.kron` of an identity with that block puts one block per pair on the diagonal.

$$B_{\max} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \\ 0 & -1 \end{pmatrix}, \quad W_1 = \begin{pmatrix} B_{\max} & 0 & \cdots & 0 \\ 0 & B_{\max} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & B_{\max} \end{pmatrix} \in \mathbb{R}^{18\times 12}$$

$$R_1 = \begin{pmatrix} 1 & 1 & -1 & 0 & 0 & 0 & \cdots \\ 0 & 0 & 0 & 1 & 1 & -1 & \cdots \\ \vdots & & & & & & \ddots \end{pmatrix} \in \mathbb{R}^{6\times 18} \quad \text{(the readout: max of each triple)}$$

Layer 2 is the same with $B_{\min} = \begin{pmatrix} 1 & -1 \\ 1 & 0 \\ -1 & 0 \end{pmatrix}$ on the three (v_{2i}, v_{2i+1}) pairs. Layer 3 reads $v = (v_0, v_1, v_2)$ and holds the three comparisons, $\max(v_0, v_1)$, and a carried copy of v_2:

$$W_3 v + b_3 = \begin{pmatrix} 1 & -1 & 0 \\ 1 & -1 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 1 & -1 \\ 1 & -1 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & -1 \end{pmatrix} v + \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Rows 0-1 are g_{01} ($\text{relu}(v_0 - v_1 + 1)$ and $\text{relu}(v_0 - v_1)$), rows 2-3 are g_{02}, rows 4-5 are g_{12}, rows 6-8 are the max gate for (v_0, v_1), rows 9-10 carry v_2. Layer 4 reads $(g_{01}, g_{02}, g_{12}, \max(v_0, v_1), v_2)$ (all of them are affine readouts of layer 3's neurons):

$$W_4 u + b_4 = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -1 \end{pmatrix} u + \begin{pmatrix} -1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

The first three rows are the AND gates; their outputs *are* the one-hot. Rows 3-5 are the final max gate. The constants $-1, 0, 1$ come from $\text{relu}(g_{01} + g_{02} - 1)$, $\text{relu}((1 - g_{01}) + g_{12} - 1)$ and $\text{relu}((1 - g_{02}) + (1 - g_{12}) - 1)$ after the "1 -" parts are moved into the bias.

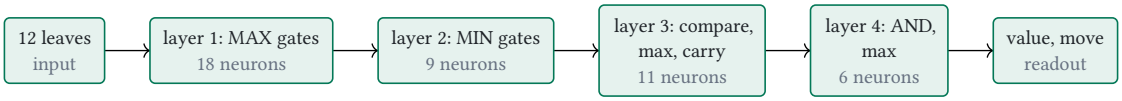


Figure 4: The whole network: 4 ReLU layers, 44 neurons. One layer per tree level, plus two for the root because it has three children.

Here is a complete forward pass for the example, with every intermediate vector:

x	`(1, 0, 3, -1,  2, 0, 2, 3,  2, 2, 3, -3)`
$h_1 = \text{relu}(W_1 x)$	`(1,0,0)(4,0,1)(2,0,0)(0,3,0)(0,2,0)(6,0,3)`
$v^{(2)} = R_1 h_1$	`(1, 3, 2, 3, 2, 3)`
$h_2 = \text{relu}(W_2 v^{(2)})$	`(0,1,0)(0,2,0)(0,2,0)`
$v = R_2 h_2$	`(1, 2, 2)`
$h_3 = \text{relu}(W_3 v + b_3)$	`(0,0, 0,0, 1,0, 0,2,0, 2,0)`
$h_4 = \text{relu}(W_4 u + b_4)$	`(0,1,0, 0,2,0)` → one-hot $(0, 1, 0)$, max $= 0 + 2 - 0 = 2$

answer	value 2, move $0 \cdot 0 + 1 \cdot 1 + 2 \cdot 0 = 1$

Alpha-beta pruning: an optimization for control flow only

Real game programs rarely evaluate every leaf. **Alpha-beta pruning** keeps two numbers, α (the best score I can already force) and β (the best score you can already force), and stops looking at a node's remaining children as soon as $\alpha \geq \beta$, because the opponent would never let the game reach here. In code it is one `break`:

```
for move in range(BRANCH[depth]):
    value, _ = alpha_beta(..., alpha, beta)
    ...
    if maximizing: alpha = max(alpha, best_value)
    else:          beta = min(beta, best_value)
    if alpha >= beta:
        break          # skip the remaining siblings
```

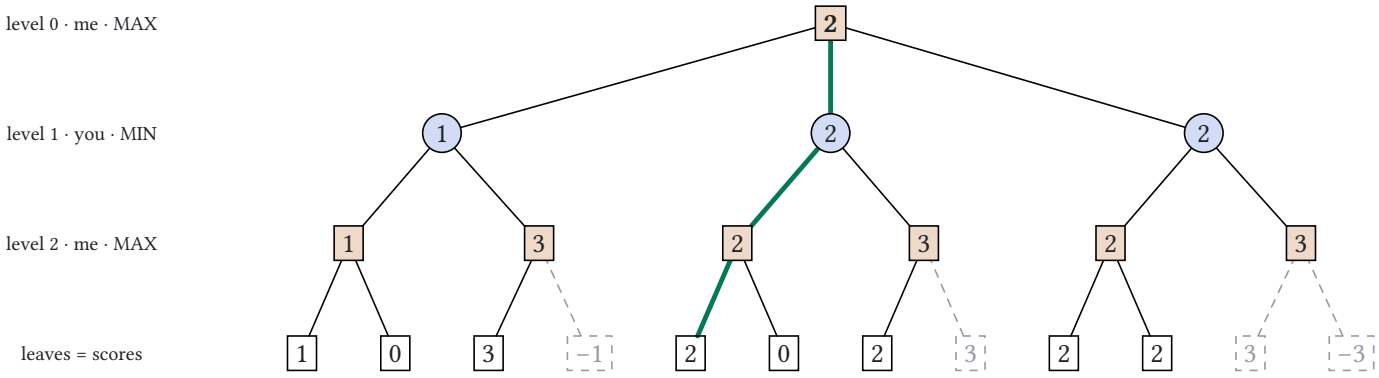


Figure 5: The same tree under alpha-beta: only 8 of the 12 leaves are looked at (dashed leaves are skipped), and the answer is identical. Leaf -1: the left MIN node already has a 1 available, and the max over $(3, \ldots)$ is at least 3, so it will never pick that branch. Leaf 3: the middle MIN node already has a 2, and the max over $(2, \ldots)$ is at least 2, so the node stays at 2. Last two leaves: the right MIN node has a 2 too, which cannot beat the 2 I already get from move 1.

A skipped leaf is a decision that depends on runtime numbers ("is $\alpha \geq \beta$ yet?"), and the network's wiring cannot change with its input. Two consequences:

- **Data flow evaluates everything.** The 12 leaves are all multiplied in, whether or not alpha-beta would have looked at them. Over the 24 106 test cases alpha-beta looked at 9.1 of 12 leaves on average; the network always looks at 12.
- **That is fine, because it is parallel.** Pruning saves *work* but not *depth*: the network still needs 4 layers however wide the tree is. A mask that turns pruned leaves into "don't care" would save no neurons, so we just do not prune.

Alpha-beta does not change the answer, only the amount of work; that is exactly why the program can check it against plain minimax (`alpha_beta(...) == minimax(...)` is asserted on every test case).

What to notice

- **A max of b numbers costs $\lceil \log_2 b \rceil$ layers.** A ReLU compares two numbers at a time. The root has three children, so it takes two rounds ($\max(\max(v_0, v_1), v_2)$), and those rounds share their layers with the comparisons that feed the argmax.
- **Ties are decided by which side is strict.** Use $[v_i \geq v_j]$ for the earlier entry and $1 - [v_i \geq v_j]$ (that is, $v_j > v_i$) for the later one, and exactly one entry survives. If you used the same non-strict test on both sides you would get two winners, and if you used the strict test on both you would get none.
- **No big-M needed here.** Every child always exists, so nothing has to be masked. The only requirement is that the leaves are integers (for the $+1$ in step); the leaf range does not matter for correctness. Chapter 5 part two needs big-M because some children do not exist.
- **Testing.** $7^{12} \approx 1.4 \times 10^{10}$ leaf assignments is too many to enumerate, so the file checks all 4096 assignments of 0/1 leaves (lots of ties), 20 000 random assignments with a fixed seed, and hand-picked cases.

Try it

1. A binary tree of depth 4 (16 leaves) alternating MAX, MIN, MAX, MIN: how many ReLU layers and how many neurons does the value computation need with the gates above?
2. What changes in the argmax if ties should go to the **highest** index?
3. For leaves $(0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1)$ what are v, the value and the move?

> **Answers**
>
> 1. One layer per level: 4 layers. Gates: $8 + 4 + 2 + 1 = 15$ pairs, 3 neurons each: **45** neurons (and no argmax).
> 2. Use the strict test for the earlier entry: $g'_{ij} = \text{step}(v_i - v_j)$ (drop the $+1$ from the bias of the first neuron of each comparison). Then $g'_{ij} = [v_i > v_j]$ and $1 - g'_{ij} = [v_j \geq v_i]$: the later entry wins ties.
> 3. Level 2: $\max = (0, 0, 1, 1, 1, 1)$; level 1: $\min = (0, 1, 1)$, so $v = (0, 1, 1)$, value 1; entries 1 and 2 tie, so the move is 1.

18 Minimax for a real game: subtraction Nim

basic_python/05_game_search/18_nim_minimax.py

The game

A pile of n objects, $0 \le n \le 12$. Players alternate. On your turn you take 1, 2 or 3 objects, never more than the pile holds. **Whoever takes the last object wins**; so a player who faces an empty pile has already lost.

Scores are from the point of view of the player who is to move at the root (me): $+1$ I win, -1 I lose, 0 not decided yet. We need both the value and my best move (how many to take; ties go to the smaller take; 0 means "no move possible").

Stage A: depth-limited minimax, and the full solve

```
def minimax(pile, depth, me_to_move):
    if pile == 0:                       # the previous player took the last one
        return (-1 if me_to_move else +1), 0
    if depth == 0:                      # out of lookahead
        return 0, 0                     # unresolved
    best_value, best_move = None, None
    for k in (1, 2, 3):
        if k > pile: break              # illegal
        value, _ = minimax(pile - k, depth - 1, not me_to_move)
        if best_move is None or (value > best_value if me_to_move else value < best_value):
            best_value, best_move = value, k
    return best_value, best_move

control_flow(n) = minimax(n, 4, True)   # looks 4 plies ahead
```

There are two Stage A functions in the file, and each data-flow version is checked against a specific one:

data-flow version	checked against	what it computes
1 · tree (`branchless`, `network`)	`control_flow(n)`: 4-ply minimax	value $\in \{-1, 0, +1\}$; 0 if undecided after 4 plies
2 · table (`branchless_table`, `network_table`)	`control_flow_full(n)`: memoized minimax to the end	value $\in \{-1, +1\}$ always

They are different functions because lookahead is limited. Here is the whole comparison, for every pile, as printed by the program:

n	4-ply value	4-ply take	full value	full take	wins?	$n \bmod 4$
0	−1	0	−1	0	no	0
1	+1	1	+1	1	yes	1
2	+1	2	+1	2	yes	2
3	+1	3	+1	3	yes	3
4	−1	1	−1	1	no	0
5	+1	1	+1	1	yes	1
6	+1	2	+1	2	yes	2
7	+1	3	+1	3	yes	3
8	−1	1	−1	1	no	0
9	**0**	1	+1	1	yes	1
10	**0**	1	+1	**2**	yes	2
11	**0**	1	+1	**3**	yes	3
12	**0**	1	−1	1	no	0

For $n \le 8$ four plies are enough to see the end of every line that matters, and the two columns agree. For $n \ge 9$ the 4-ply search cannot tell, and says 0; its "best move" is just the first one. The rule is general and the program asserts it: **a lookahead value of ± 1 is always the true value** (a forced win or loss inside the horizon is a real win or loss); 0 only means "unknown". Its best move can differ from the full solve's only when the value is 0.

(The pattern in the last columns is the classical result: the player to move loses exactly when n is a multiple of 4. Taking $n \bmod 4$ objects is then the unique winning move.)

Data-flow version 1: the tree

A tree whose shape never changes

Write node (j, i) for "after j plies, along path i". The path is the base-3 digits of i, digit d meaning "take $d+1$". The children of (j, i) are $(j+1, 3i), (j+1, 3i+1), (j+1, 3i+2)$. So *every* level j has exactly 3^j nodes, for every input, including nodes that could not exist in a real game. The pile at a node is

$$\text{pile}(j, i) = n - (\text{total taken along the path})$$

which is **affine in n**: a constant offset subtracted from the input. Who moves is also fixed: me on even levels (MAX), you on odd levels (MIN).

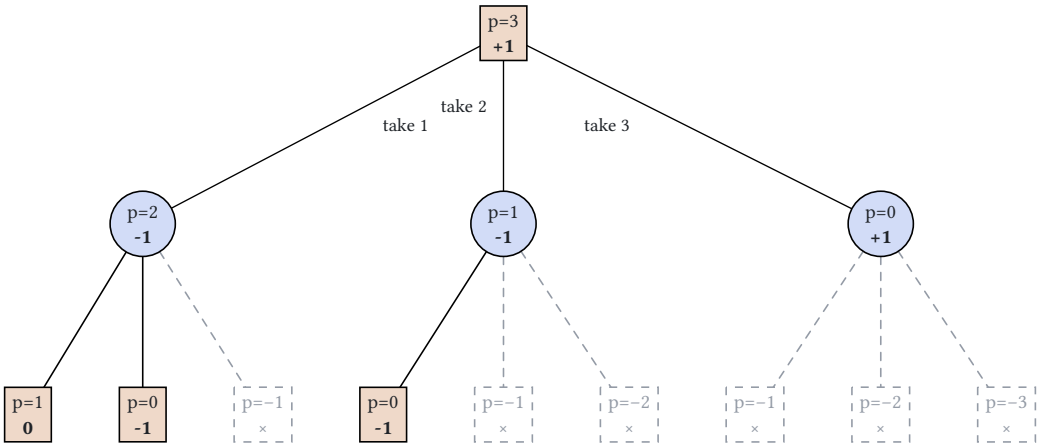


Figure 6: The fixed tree for $n = 3$ with 2 plies of lookahead (9 leaves; $D = 4$ has 81). Dashed nodes have a negative pile: that move does not exist. The node for "take 3" has pile 0: the game is over and the player who faces it (you) has lost, so it is worth $+1$ to me.

The two masks

For a node with pile p we need two 0/1 numbers, both from the toolbox:

$$\text{legal} = \text{step}(p+1) \quad (\text{pile} \ge 0), \qquad \text{zero} = 1 - \text{step}(p) - \text{step}(-p) \quad (\text{pile} = 0).$$

Why "legal" is just $p \ge 0$. A move "take k" is legal iff $k \le p_{\text{parent}}$, i.e. iff the new pile $p_{\text{parent}} - k \ge 0$. And once the game has ended (pile 0), taking anything gives a negative pile. Piles strictly decrease along a path, so $p \ge 0$ at a node implies the same for all its ancestors, and no extra "was the game already over?" mask is needed.

Numbers for the tree above ($n = 3$, $D = 2$):

node	pile p	legal	zero	type	value
root (take nothing)	3	1	0	MAX	+1
take 1	2	1	0	MIN	−1
take 3	0	1	1	MIN	+1
take 1, take 3	−1	0	0	leaf	garbage: masked away
take 1, take 2	0	1	1	leaf	−1

Terminal positions without an `if`

A node with pile 0 has no children to look at; its value is fixed: -1 if it is my turn (I have lost), $+1$ if it is yours. The trick is to give every inner node a **fourth entry** in its max/min, the "no move" entry, worth

$$e_0 = \text{fill} + s \cdot \text{zero}$$

where $s = +1$ on MAX levels, -1 on MIN levels, and $\text{fill} = -2s$ (the value that loses every comparison: -2 for max, $+2$ for min). On a MAX level with zero $= 1$ that is $-2 + 1 = -1$: "I have lost". On a MIN level it is $2 - 1 = +1$. When the pile is not 0, $e_0 = \text{fill}$ and it is ignored.

Leaves (level D) have no children at all: value $= -s \cdot \text{zero}$, i.e. -1 or $+1$ if the game just ended there and 0 ("undecided") otherwise.

Keeping impossible children out: the big-M select

The other three entries are the children, take 1, take 2, take 3. A child that does not exist must not win the max, so we replace its value by `fill`:

$$e_k = \text{fill} + \text{select}(\text{legal}_k, v_k - \text{fill}), \quad k = 1, 2, 3$$

with the README's big-M select $\text{select}(m, x) = \text{relu}(x - M(1 - m)) - \text{relu}(-x - M(1 - m))$, which equals x when $m = 1$ and 0 when $m = 0$, as long as $|x| \le M$. Then $e_k = v_k$ for a legal child and $e_k = \text{fill}$ for an illegal one.

Two numbers for node "take 1" of the figure (a MIN level, $s = -1$, $\text{fill} = +2$):

- child "take 2" (pile 0, legal, value -1): $x = -1 - 2 = -3$, $\text{select}(1, -3) = \text{relu}(-3) - \text{relu}(3) = -3$, so $e = 2 + (-3) = -1$.
- child "take 3" (pile -1, illegal, garbage value 0): $x = 0 - 2 = -2$, $\text{select}(0, -2) = \text{relu}(-2 - 4) - \text{relu}(2 - 4) = 0 - 0 = 0$, so $e = 2 + 0 = 2$ (= fill).

Where $M = 4$ comes from. The select is exact only if $|x| \le M$ for *every* child, including the nonexistent ones, so we must bound the garbage too. Leaves are in $\{-1, 0, +1\}$. Every entry is either `fill` (± 2) or the value of a real child (in $[-1, 1]$); a max or min of such entries is again in $[-2, 2]$. So all node values, real or not, lie in $[-2, 2]$, and $|v - \text{fill}| \le 2 + 2 = 4$. That is $M = 4$. With $M = 3$ the select would break: a garbage value -2 below a MIN level ($\text{fill} = +2$) gives $x = -4$ and $\text{select}(0, -4) = \text{relu}(-4 - 3) - \text{relu}(4 - 3) = -1$ instead of 0, which leaks into the answer; the tests catch this immediately.

> **Why not just subtract M from illegal children?**
>
> It looks cheaper to use $v - M(1 - \text{legal})$ for max (no ReLUs). But then a garbage value flows upward through levels and *grows* by M at each one, so no fixed bound holds and no fixed M works. The select pins every illegal child to exactly `fill`, which keeps all values in $[-2, 2]$.

Combining: three layers per level

For one level j (with $g = 3^j$ nodes), working from the level below:

layer	neurons	what it does
SELECT	6 per node (3 children × 2), 5 per node (flags), + 1 (carried n)	the 3 selects; flags of level j; carry n
MAX/MIN 1	6 per node, + 1 (carried n)	entries (e_0, e_1) and (e_2, e_3) through a 3-neuron gate each
MAX/MIN 2	3 per node, 5 per node (flags), + 1 (carried n)	the two results through one more gate; flags of level j again for the parent's select

Two details deserve a sentence each.

- **Flags are recomputed, not carried.** The flags ("legal", "zero") are functions of n alone: five ReLUs per node, $\text{relu}(p + 1), \text{relu}(p), \text{relu}(p - 1), \text{relu}(-p), \text{relu}(-p - 1)$ with $\text{step}(p + 1) = \text{relu}(p + 1) - \text{relu}(p)$ and so on. They are computed in the layer just before they are needed, and n itself is carried from layer to layer by a single

neuron $\mathrm{relu}(n) = n$ (exact because $n \geq 0$). Carrying a few numbers is much cheaper than carrying hundreds of flags.

- **Nothing multiplies two runtime numbers.** The masks enter only via the select (a ReLU of an affine function of the mask and the value), never as `mask * value`.

The root (level 0) is a MAX level with four entries $e_0..e_3$ (take nothing, take 1, 2, 3), and needs the argmax. With four entries there are six comparisons $g_{ij} = [e_i \geq e_j]$, $i < j$, each two ReLUs, and the winner for entry i is an AND of three bits, $\mathrm{relu}(\mathrm{sum} - 2)$:

$$\mathrm{AND} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 1 & 0 \\ 0 & -1 & 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 & -1 & -1 \end{pmatrix}, \quad b = \begin{pmatrix} -2 \\ -1 \\ 0 \\ 1 \end{pmatrix}$$

(the columns are the pairs $01, 02, 03, 12, 13, 23$). A $+1$ means "entry i must be $\geq$ entry j"; a -1 means "entry j must be strictly better than entry i", which is the $1 - g$ moved into the bias. The one-hot decodes to the take: $\mathrm{take} = \text{one-hot} \cdot (0, 1, 2, 3)$. Since entry 0 is "no move", it only wins at $n = 0$, where all other entries equal `fill`.

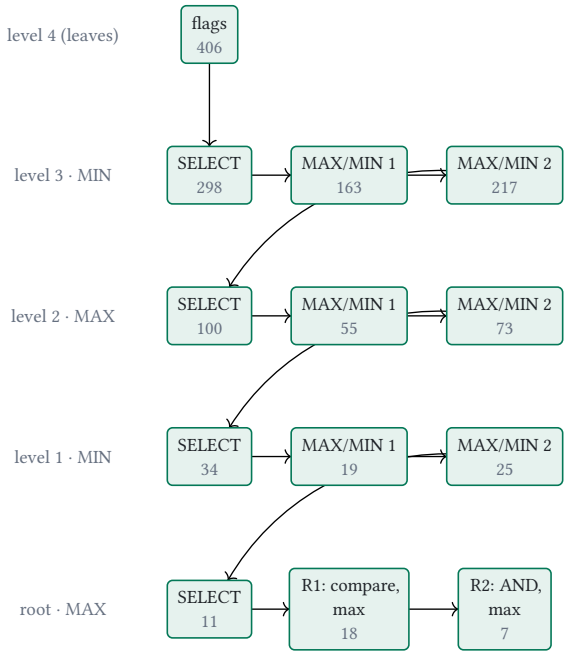


Figure 7: The tree network for $D = 4$: 13 ReLU layers, 1426 neurons. Each row is one tree level, computed bottom-up; each level costs three layers.

A worked example: $n = 7$

With $D = 4$ the root's entries come out as $e = (-2, -1, -1, +1)$: no move (-2, because the pile is not 0); taking 1 leaves 6, a won pile for the player to move (you), so worth -1 to me; taking 2 leaves 5, also -1; taking 3 leaves 4, a lost pile for you, so $+1$. The six comparisons $g_{ij} = [e_i \geq e_j]$ in the order $01, 02, 03, 12, 13, 23$ are $(0, 0, 0, 1, 0, 0)$. Feeding them to the AND matrix and bias above:

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 1 & 0 \\ 0 & -1 & 0 & -1 & 0 & 1 \\ 0 & 0 & -1 & 0 & -1 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} -2 \\ -1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -2 \\ 0 \\ -1 \\ 1 \end{pmatrix}, \quad \mathrm{relu}(\ldots) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

One-hot $(0, 0, 0, 1)$: take 3, value $\max(e) = +1$. (Check: $7 \bmod 4 = 3$.)

For $n = 12$ every child is unresolved within the horizon: all three have value 0, the root entries are $(-2, 0, 0, 0)$, entries 1, 2 and 3 tie, and the lowest index, take 1, wins. Value 0: "I cannot tell."

Data-flow version 2: the table

The tree recomputes the same sub-positions again and again: from 12, taking 1 then 2 and taking 2 then 1 both reach 9. The pile is the whole state. So solve each pile **once**, bottom-up. Let $\text{win}[p] = 1$ if the player to move with p left wins with perfect play. Then

$$\text{win}[p] = \text{OR}_{k \le p}\, \text{NOT win}[p-k], \quad \text{win}[0] = 0,$$

(a position wins if some move leaves the opponent in a lost position), where "OR" is $\text{step}(\text{sum})$ and "NOT" is $1 - x$. None of this depends on the input n, so it is a computation on *constants*, done once when the file loads:

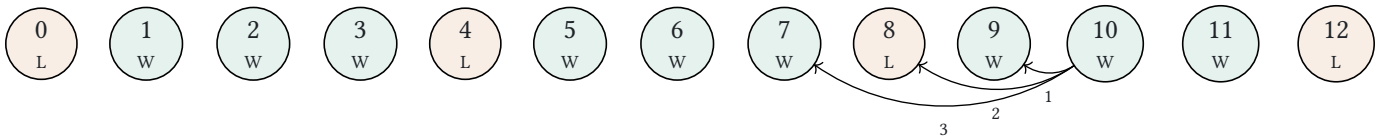


Figure 8: The 13 positions (W = player to move wins, L = loses). Position 10 depends on 9, 8 and 7 only, so win[10] = (NOT win[9]) OR (NOT win[8]) OR (NOT win[7]) gives 0 OR 1 OR 0 = 1. The winning take is 2: it leaves the lost position 8.

The build also records the best take: the smallest k that reaches a lost position, or take 1 if there is none, or 0 at $p = 0$. The resulting constants:

p	0	1	2	3	4	5	6	7	8	9	10	11	12
value	−1	+1	+1	+1	−1	+1	+1	+1	−1	+1	+1	+1	−1
take	0	1	2	3	1	1	2	3	1	1	2	3	1

Looking up n in a constant table needs a one-hot of n (Chapter 3: with a constant table, `onehot @ table` is a plain matrix product). Equality bits would cost four ReLUs per position; the **hat** trick needs only one shared ReLU per position. With $r_t = \text{relu}(n - t)$,

$$\text{onehot}_i = r_{i-1} - 2r_i + r_{i+1}.$$

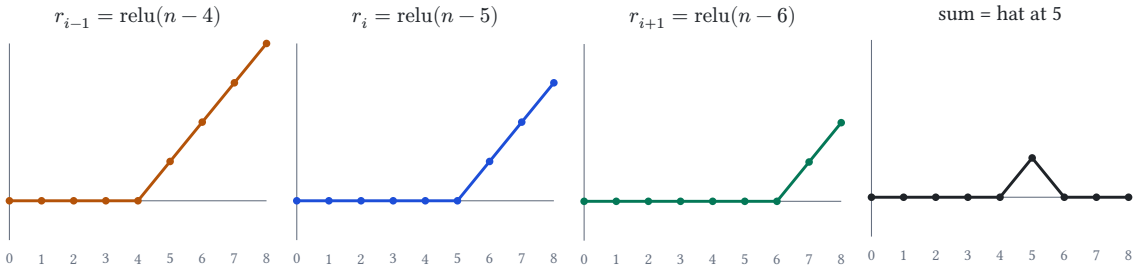


Figure 9: Three shifted ReLUs add up to a spike at $n = 5$: the sum is 0 for $n \le 4$ (all three are 0), 1 at $n = 5$, and 0 again for $n \ge 6$ because the slopes $+1, -2, +1$ cancel. For integers this is an exact one-hot.

The whole network is one ReLU layer and two matrices:

$$r = \text{relu}\left(\begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} n + \begin{pmatrix} 1 \\ 0 \\ -1 \\ \vdots \\ -13 \end{pmatrix} \right), \quad \text{onehot} = Hr, \quad \begin{pmatrix} \text{value} \\ \text{take} \end{pmatrix} = T\,\text{onehot}$$

with $r \in \mathbb{R}^{15}$ ($t = -1..13$), $T \in \mathbb{R}^{2\times 13}$ the table above, and $H \in \mathbb{R}^{13\times 15}$ the hat readout:

$$H = \begin{pmatrix} 1 & -2 & 1 & 0 & 0 & \cdots \\ 0 & 1 & -2 & 1 & 0 & \cdots \\ 0 & 0 & 1 & -2 & 1 & \cdots \\ \vdots & & & & & \ddots \end{pmatrix}$$

15 neurons, 1 layer, and it answers the *full* game, not 4 plies of it. It is checked against `control_flow_full(n)`: value and take agree for all 13 piles.

Tree versus table

lookahead D	nodes in tree	layers	neurons	unresolved piles $n \le 12$
1	4	4	52	4 .. 12
2	13	7	160	5 .. 12
3	40	10	478	8 .. 12
4	121	13	1 426	9 .. 12
5	364	16	4 264	12
6	1 093	19	12 772	none
12	797 161	37	≈ 9.3 million	none (longest game from 12 has 12 plies)
20	≈ 5.2 billion	61	≈ 6.1×10^{10}	none (exact for piles up to 20)

versus the table: $N + 3 = 15$ neurons for $N = 12$, or 23 for $N = 20$, 1 layer, exact.

The tree grows as 3^D because it counts *paths*; the table grows as N because it counts *distinct positions*. (This is the neural counterpart of adding `@cache` to the recursive Stage A function, which is exactly what `solve()` does.) Notice what the table *is*: the answer, precomputed. Its weights contain the solution of the game, found by running the recurrence offline; the tree network actually *searches*. Use a table when positions repeat and there are few of them; use the tree when the position is too rich to tabulate.

What to notice

- **Illegal is not the same as zero.** Replacing an illegal child's value by 0 would let it win a `max` whenever the real values are negative. It must be replaced by something that loses (`fill`), and which something depends on the level: -2 at MAX, $+2$ at MIN.
- **Bound the garbage.** `select` needs $|x| \le M$ for the cases where the mask is 0, which are exactly the cases where x is garbage. Derive the range of *every* value in the network, real or not, before picking M.
- **Perspective.** All values are from the root player's view (so MAX levels are mine, MIN levels yours). Terminal values then depend on the parity of the level, a constant.
- **"0" means "unknown", not "draw".** The depth limit returns 0; a game with genuine draws would need a different marker.
- **The root is special only for the argmax.** The values at the root come from the same three-layer pattern as every other level.
- **Test on tiny inputs.** $n = 0, 1, 2$ are where nearly all nodes are illegal; a wrong mask shows up there first. The program checks every n from 0 to 12.

A recipe for your own game search

1. **Fix the shape.** Pick the maximum branching b and lookahead D. The tree has b^D leaves whatever the position.
2. **Make positions affine in the input.** If the game state is a number, the state at a node is $x - \text{offset}(j, i)$ for a constant offset; if it is a vector (a board), each node's board is the root board plus a constant per move. Write the offsets down as constant arrays, generated at import time from the tree structure.
3. **Make the masks.** "Is this move legal?" and "is the game over here?" must each become a 0/1 number: step of an affine expression (as $\text{step}(p + 1)$ here), or a product of such masks via AND (as $\text{relu}(a + b - 1)$). Check whether illegality of an ancestor already implies illegality of the descendant (here it does, because piles only shrink); if not, AND the parent's mask in.
4. **Give every inner node a "no move" entry** so that game-over positions need no special case, and give every leaf its terminal score (or a heuristic score; any affine-plus-ReLU function of the position works as a leaf evaluation, that is what a depth cutoff in a real engine is).
5. **Fill, don't zero.** Mask child values with a big-M select toward `fill` ($-$BIG on MAX levels, $+$BIG on MIN levels). Compute the range of all values to choose M.
6. **Combine with gates.** Max/min of m entries: a tournament of 3-neuron gates, $\lceil \log_2 m \rceil$ layers. If all values are known non-negative, 2 neurons per gate is enough.
7. **Argmax at the root only.** Compare entries pairwise, AND them into a one-hot, decode the move with a constant vector. Decide tie-breaking by which side of each comparison is strict.
8. **Check against a plain recursive version on every input you can.** Include tiny and extreme inputs.

9. **Watch the size.** About b^D nodes times a few neurons. If the same sub-positions appear many times, build a table (dynamic programming) as in version 2.

Alpha-beta, iterative deepening and move ordering are all ways of doing less *work* in control flow; in a network the width is free and the depth is $O(D)$, so you do not need them. What you need instead is to keep b^D small enough.

Try it

1. Change the rules so a move may take 1, 2 or 4. Which lines of version 2 change, and which piles lose?
2. Which lookahead D makes version 1 resolve every $n \leq 12$? How many neurons?
3. How many neurons would version 1 need to solve $n = 20$ completely, and version 2?

> **Answers**
>
> 1. Only `TAKES = (1, 2, 4)` changes (and `build_table` already loops over it). The losing piles are $0, 3, 6, 9, 12$, the multiples of 3. (Version 1 would also need the offsets $1, 2, 4$ in `level_offsets`, the `piles` loop, and the `E_CHILD` matrix.)
> 2. $D = 6$: the program, run with $D = 6$, finds no unresolved pile. The network then has 12 772 neurons; with $D = 5$ only $n = 12$ is unresolved.
> 3. $D = 20$ needs `tree_neurons(20)` $\approx 6.1 \times 10^{10}$ neurons. Version 2 needs $20 + 3 = 23$.